Policy gradient on the walking game

Live demo: charith-reddy-pareddy.github.io/dog-game

Main research report: RESEARCH_REPORT.md

Code: github.com/Charith-Reddy-Pareddy/dog-game

This report asks one question: how well do policy-gradient methods, REINFORCE, A2C and PPO, learn the walking game? The walking game is the angle-radius version of the dog game, where each player's action is a direction and a step length. It is separate from the main research report, which covers the whole project, and it says how many runs each claim rests on.

Summary

- **All three algorithms learn to walk to the corners, but not equally reliably.** After 300 episodes A2C has all 5 seeds at the one-shot Nash corners and 34 of its 35 later checkpoints there. REINFORCE has 4 of 5 seeds and 24 of 35. PPO has 2 of 5 and 14 of 35 (Section 4.1).
- **PPO is the fastest to get there and the worst at staying.** It has the lowest distance after 50 and 100 episodes, but three of its five seeds leave the corners again later (Section 4.1).
- **A smaller learning rate does not fix PPO. One epoch does, given three times more training.** One-epoch PPO has 5 of 5 seeds at the corners after 900 episodes, while default PPO still has two seeds stuck far away (Sections 4.3 and 4.4).
- **A lower discount makes the game easier to learn and a higher one harder.** A longer step costs nothing and a shorter one is slower (Section 4.5).
- **The ranking holds on other house setups, but some setups are much harder.** On three other setups A2C is again the most reliable wherever any algorithm learns the game, PPO is the least reliable or tied for least, and on one setup none of them gets more than 1 of 5 seeds to the corners in 300 episodes. Dog inertia slows learning (Section 4.7).
- **Scores are noisy.** Evaluating a policy changes the random numbers in training, so the paths are sensitive, and a single end-of-training number would mislead (Section 4.6).
- **Not shown:** that the one-shot corners are an equilibrium of the walking game itself. The game has not been solved (Section 6).

1. The walking game

In the main dog game a player picks any point in the unit square and the dog moves to the weighted average of the two picks. In the walking game each player has a position and walks. Every round each player picks a direction (an angle) and a step length of up to 0.2, and moves that way from where it stands, stopping at the edge of the square. The dog then moves to the average of the two new positions, and each player is rewarded by the negative squared distance from the dog to its own house. This is my reading of "angle-radius action", described in ASSUMPTIONS.md.

Item	Value
Houses	red at (0.88, 0.12), blue at (0.3, 0.65)
Weight	the dog goes to 0.5 * red + 0.5 * blue
Start	each player on its own house, the dog between them

Item	Value
Action	an angle and a step fraction in [0, 1], scaled by the maximum step 0.2
Episode	15 steps, discount 0.9
Reward	minus the squared distance from the dog to the player's own house
State	the dog's position and both players' positions (6 numbers)
Reference	the one-shot Nash corners: red at (1, 0), blue at (0, 1)

Why those corners. In the one-shot game each player overshoots toward the edge to counteract the other. With these houses the equilibrium has red at the bottom-right corner and blue at the top-left, and the dog ends in the middle. The walking game is judged against it: a player has "walked to its corner" when it ends there.

How it is scored. The distance is the sum of red's and blue's distances from their corners at the end of a 15-step game, averaged over 10 sampled games. A seed is **at the corners** when that distance is below 0.1, which is my threshold. Policies are scored every 25 episodes, and the **late checkpoints** are those from 150 episodes on (7 per seed, so 35 for five seeds).

2. The policy

Each player has its own small network that reads the state and outputs a distribution over a move (`doggame/polar_agent.py`):

- **The angle** comes from a von Mises distribution, whose mean direction is `atan2` of two network outputs and whose concentration is a third output passed through softplus plus 1. A von Mises treats 0 and 2 pi as the same direction, so there is no wraparound problem.
- **The step** comes from a Beta distribution on [0, 1], with both shape parameters a softplus output plus 1, scaled by the maximum step. Both parameters are at least 1, so the distribution has a single peak.
- **Exact gradients.** Both distributions have exact log-probabilities, so nothing is clipped before the gradient is taken. This differs from the main game's Gaussian policy, whose actions are clipped to the square.
- **Size.** Each player's network is 6 inputs, one hidden layer of 16 tanh units and 5 outputs. A2C and PPO add a value network per player (6 inputs, two hidden layers of 32 tanh units, 1 output).

3. The algorithms

- **REINFORCE** weights each action's log-probability by the discounted return minus the mean return over the episode.
- **A2C** replaces the mean-return baseline with the learned value of the state.
- **PPO** is A2C with each episode reused for 4 epochs, the probability ratio clipped at 0.2 and the advantages normalized (`doggame/train_ac.py`).

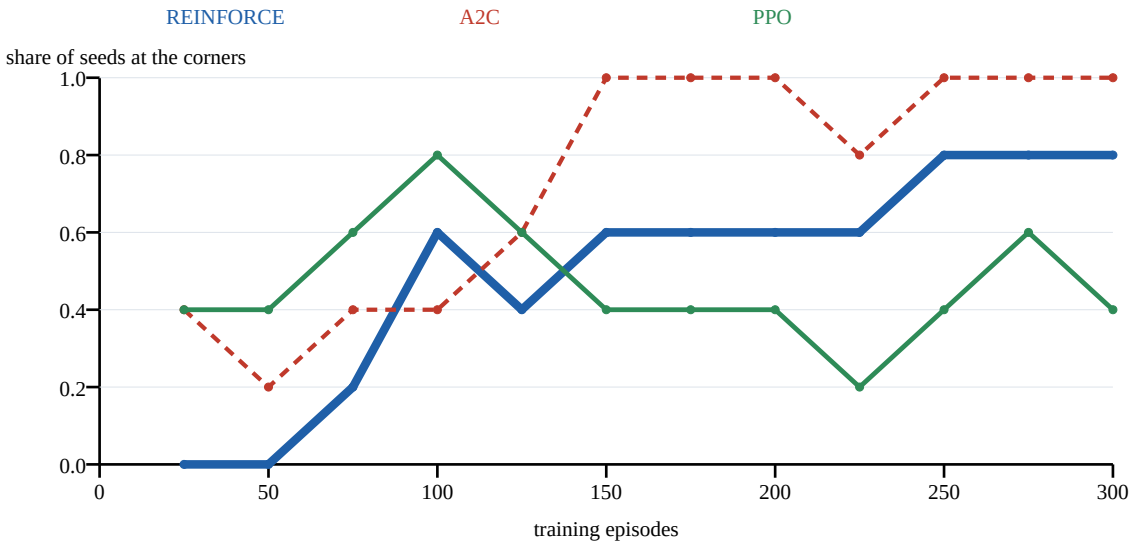
All three use Adam at learning rate 0.01 and one self-play episode per update, with both players learning from the same episode. Training is 300 episodes unless stated, over seeds 0 to 4, and the policy is scored every 25 episodes by 10 sampled games. There is no entropy bonus.

4. Results

Every number comes from `results/dog_game_studies.jsonl` (the `walking`, `ppo_variants`, `ppo_long`, `walking_settings`, `walking_houses` and `walking_inertia` studies) and `results/dog_game_runs.jsonl`, and the tables are produced by `doggame/dog_game_summary.py`.

4.1 Three algorithms, 300 episodes

The chart shows the share of the five seeds at the corners after each number of episodes. In the table, "At N" is the mean distance after N episodes.

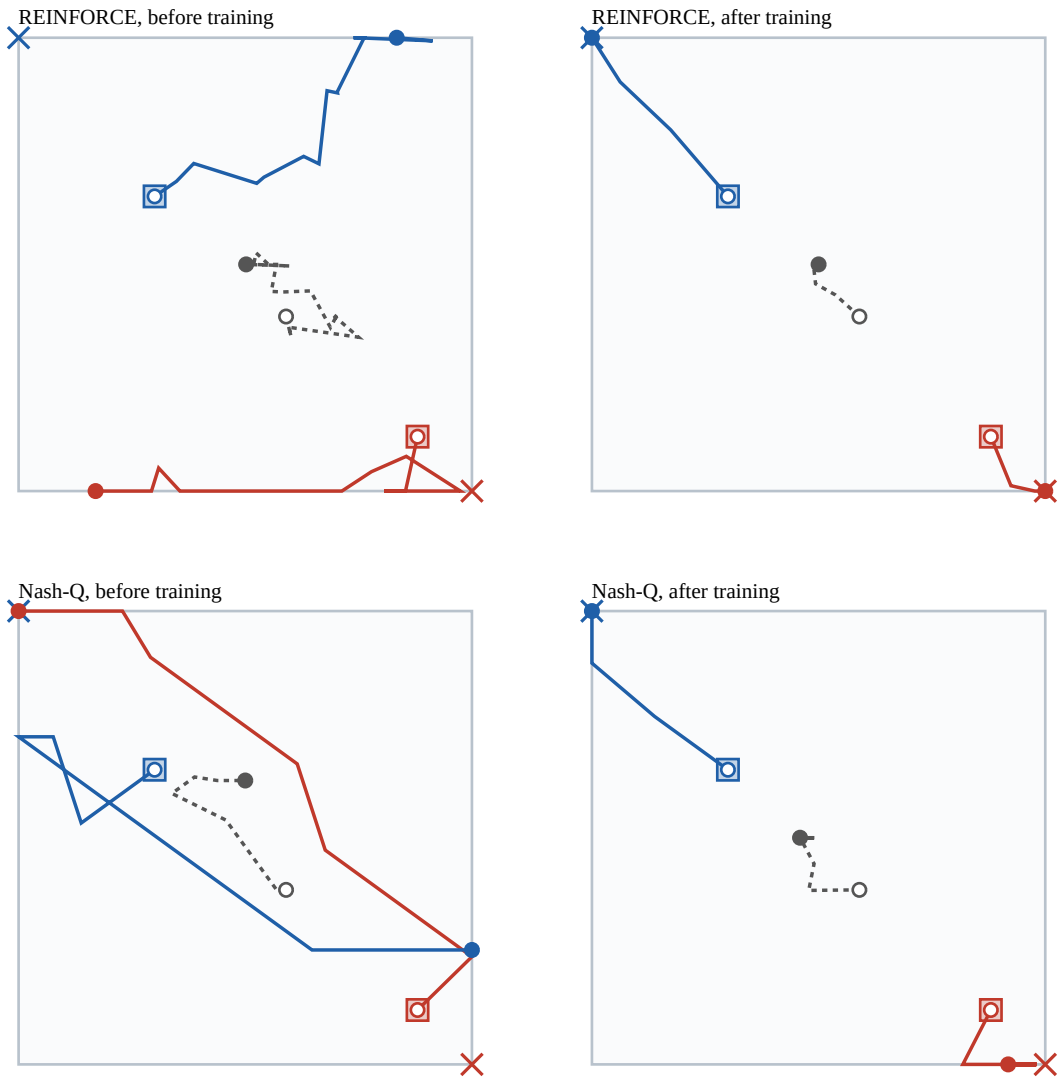


Algorithm	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
REINFORCE	1.49	0.696	0.341	0.130	0.055	24 of 35	4 of 5
A2C	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5
PPO	1.49	0.263	0.127	0.500	0.371	14 of 35	2 of 5

- **A2C is the most reliable.** All 5 seeds end at the corners, and 34 of its 35 later checkpoints are under 0.1.
- **REINFORCE is slower and more variable.** Its mean distance is 0.34 after 100 episodes against 0.20 for A2C, it has 24 of 35 later checkpoints at the corners, and one seed stays near 0.2.
- **PPO gets there first and does not stay.** It has the lowest distance after 50 and 100 episodes (0.26 and 0.13), but seeds 1, 2 and 4 leave the corners again later, with their distance climbing back to about 1.0, which means a player has left its corner. It has 14 of 35 later checkpoints at the corners.
- **This is the opposite of the main game**, where PPO was the fastest and most reliable (Section 5.2 of the main report).

4.2 What the paths look like

One sampled 15-step game for seed 0, before and after training. REINFORCE is the policy-gradient method shown; the Nash-Q network is included as a reference.



square = house, X = one-shot Nash corner, open circle = start, filled circle = end; dashed gray = dog

Before training the players wander. After training both walk to their corners and the dog settles near the middle. The live site lets you pick a seed and step through these games with a play button. For reference, the earlier five-seed runs of REINFORCE (300 episodes) and the 10-direction Nash-Q network (60 episodes) gave:

Method	Distance before training	Distance after training	Seeds that improved
REINFORCE (angle and step)	1.49	0.04	5 of 5
Nash-Q (10 directions)	1.95	0.01	5 of 5

4.3 Can PPO be made more reliable?

PPO with one epoch and/or a learning rate of 0.003 instead of 0.01, five seeds each, scored as above:

Setting	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
A2C (reference)	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5
PPO, lr 0.01, 4 epochs (default)	1.49	0.263	0.127	0.500	0.371	14 of 35	2 of 5
PPO, lr 0.01, 1 epoch	1.49	0.595	0.479	0.249	0.123	21 of 35	4 of 5
PPO, lr 0.003, 4 epochs	1.49	0.477	0.361	0.448	0.362	15 of 35	1 of 5
PPO, lr 0.003, 1 epoch	1.49	0.836	0.641	0.470	0.322	8 of 35	2 of 5

- **Fewer epochs helps.** One epoch at learning rate 0.01 has 4 of 5 seeds at the corners and 21 of 35 later checkpoints there, against 2 of 5 and 14 of 35 for the default. It is slower early (mean distance 0.60 after 50 episodes, against 0.26).
- **A smaller learning rate does not help.** It gives 1 of 5 seeds with 4 epochs and 2 of 5 with one epoch.
- **None matches A2C within 300 episodes.**

4.4 Training three times longer

A2C and the two PPO settings that matter, 900 episodes, scored every 25 episodes (late checkpoints are from 450 episodes on):

Setting	Before training	At 300	At 600	At 900	Late checkpoints at the corners (from 450)	Seeds at the corners at the end
A2C	1.49	0.011	0.003	0.003	95 of 95	5 of 5
PPO, 4 epochs (default)	1.49	0.371	0.499	0.669	50 of 95	3 of 5
PPO, 1 epoch	1.49	0.123	0.033	0.007	77 of 95	5 of 5

- **A2C stays at the corners**, at every late checkpoint and in all five seeds.
- **One-epoch PPO settles.** It ends with 5 of 5 seeds at the corners, though it takes longer than A2C (mean distance 0.12 after 300 episodes against 0.011).
- **Default PPO does not settle.** It ends with 3 of 5 seeds at the corners and 50 of 95 late checkpoints. Two seeds leave the corners and do not return within 900 episodes (their distance is about 1.0 and over 2 at the end), so the instability is not just a slow start.
- The two settings with a learning rate of 0.003 were not run longer.

4.5 A different step length or discount

A2C with the maximum step at 0.1 or 0.4, or the discount at 0.5 or 0.99, five seeds each, 300 episodes, next to the default (0.2 and 0.9):

Setting	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
max step 0.1, discount 0.9	1.08	0.439	0.386	0.234	0.184	20 of 35	3 of 5

Setting	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
max step 0.2, discount 0.9 (default)	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5
max step 0.4, discount 0.9	1.67	0.509	0.182	0.036	0.028	32 of 35	5 of 5
max step 0.2, discount 0.5	1.49	0.409	0.088	0.009	0.003	35 of 35	5 of 5
max step 0.2, discount 0.99	1.49	0.836	0.442	0.102	0.056	22 of 35	4 of 5

- **A lower discount makes learning faster and a higher one slower.** A discount of 0.5 gets 5 of 5 seeds to the corners and 35 of 35 late checkpoints, and 0.99 gets 4 of 5 and 22 of 35.
- **A longer step costs nothing and a shorter one is slower.** A step of 0.4 is as reliable as the default (5 of 5, 32 of 35), and 0.1 gets 3 of 5 seeds and 20 of 35 late checkpoints within 300 episodes. More episodes might close that gap, which was not run.
- **These change how fast the players get to the corners, not where.** The one-shot Nash corners do not depend on the step length or the discount.

4.6 Scores depend on how they are measured

A first run of this comparison was scored only at 50, 100, 200 and 300 episodes. It gave the same order of algorithms but different outcomes for individual seeds (PPO seed 3 ended at 1.4 there and at 0.0 in the run above), because evaluating a policy samples games and so draws random numbers, which changes the rest of training. The paths are sensitive. A trace of one PPO seed showed its distance moving between 0.0 and 1.0 and back during training. That is why this report scores every 25 episodes and reports how many of the later checkpoints are at the corners, not just the last one.

4.7 Other house setups and dog inertia

Other house setups. The three algorithms on the default setup and on three others (the first three house setups of the main game), five seeds each, 300 episodes. Each setup has its own one-shot Nash corners, and the score is the distance from those:

House setup	Algorithm	Before training	At 100	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
default: (0.88, 0.12) / (0.3, 0.65), w = 0.5	REINFORCE	1.49	0.341	0.055	24 of 35	4 of 5
default: (0.88, 0.12) / (0.3, 0.65), w = 0.5	A2C	1.49	0.199	0.011	34 of 35	5 of 5
default: (0.88, 0.12) / (0.3, 0.65), w = 0.5	PPO	1.49	0.127	0.371	14 of 35	2 of 5
(0.9, 0.2) / (0.1, 0.8), w = 0.5	REINFORCE	1.41	0.569	0.168	14 of 35	2 of 5
(0.9, 0.2) / (0.1, 0.8), w = 0.5	A2C	1.41	0.275	0.070	22 of 35	3 of 5
(0.9, 0.2) / (0.1, 0.8), w = 0.5	PPO	1.41	0.836	0.515	14 of 35	2 of 5

House setup	Algorithm	Before training	At 100	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
(0.8, 0.8) / (0.2, 0.3), w = 0.3	REINFORCE	1.31	0.707	0.239	18 of 35	4 of 5
(0.8, 0.8) / (0.2, 0.3), w = 0.3	A2C	1.31	0.429	0.036	29 of 35	5 of 5
(0.8, 0.8) / (0.2, 0.3), w = 0.3	PPO	1.31	0.974	0.932	9 of 35	1 of 5
(0.7, 0.1) / (0.6, 0.9), w = 0.7	REINFORCE	1.55	0.897	0.848	2 of 35	0 of 5
(0.7, 0.1) / (0.6, 0.9), w = 0.7	A2C	1.55	0.842	0.688	4 of 35	0 of 5
(0.7, 0.1) / (0.6, 0.9), w = 0.7	PPO	1.55	0.838	1.153	4 of 35	1 of 5

- **A2C is again the most reliable wherever learning works.** It has the most late checkpoints at the corners on the default setup (34 of 35) and on the (0.9, 0.2) / (0.1, 0.8) and (0.8, 0.8) / (0.2, 0.3) setups (22 and 29 of 35, against 14 and 18 for REINFORCE and 14 and 9 for PPO).
- **PPO is the least reliable, or tied for least, on the three setups where learning works.** Its early lead on the default setup does not carry over: on the other two it has the highest mean distance after 100 episodes.
- **Some setups are much harder.** On (0.7, 0.1) / (0.6, 0.9) with w = 0.7 no algorithm gets more than 1 of 5 seeds to the corners in 300 episodes, and the mean distances at the end are 0.69 to 1.15. Whether more episodes would fix it was not run.

Dog inertia. A2C when the dog keeps part of its old position each round (`w0` of 0.3 and 0.6), on the default setup:

Setting	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
w0 = 0 (default)	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5
w0 = 0.3	1.50	0.880	0.459	0.088	0.015	31 of 35	5 of 5
w0 = 0.6	1.51	0.995	0.513	0.187	0.114	25 of 35	4 of 5

- **Inertia slows learning**, as it did on the main game. The mean distance after 100 episodes is 0.20, 0.46 and 0.51 at `w0` of 0, 0.3 and 0.6, and 5, 5 and 4 of 5 seeds end at the corners.
- The one-shot corners do not depend on `w0`, since they are the steady state, so this is about how fast the players get there.
- Inertia was tried with A2C only.

5. What this does and does not show

- **Four house setups, one of them much harder.** Most runs use red at (0.88, 0.12), blue at (0.3, 0.65), w = 0.5 and no dog inertia. Section 4.7 adds three other setups for all three algorithms, and inertia for A2C only.

- **Five seeds per setting.** The counts are out of five, so small differences between settings are patterns, not a ranking. The step-length and discount study uses A2C only.
- **The corners are the one-shot equilibrium.** Heading to them shows the players learning a sensible behavior, not that they play an equilibrium of the walking game. That game has not been solved.
- **My choices.** The 0.1 threshold, the 300-episode budget, the learning rate and the network sizes were not tuned for each algorithm.
- **Why the ranking differs from the main game is not tested.** The causes of PPO's instability here are unknown; reusing each episode seems to contribute, but that is a hypothesis.

6. Open questions

These are the walking-game questions in RESEARCH_QUESTIONS.md:

- **Q23.** Is the one-shot corner pair an equilibrium of the walking game?
- **Q27.** Why is PPO unstable here? (A smaller learning rate does not help and fewer epochs do.)
- **Q30.** Do these findings hold on other house setups and with dog inertia? Partly answered (Section 4.7); other step lengths, PPO with inertia and longer training on the hard setup were not run.
- **Q31.** Does an entropy bonus or a larger network change PPO's instability?

7. Reproducing

```
python3 -m doggame.dog_game_studies walking ppo_variants ppo_long walking_settings walking_houses walking_inertia # the runs (resumable, shardable) python3 -m doggame.dog_game_summary # print every table python3 -m doggame.research_report_pdf WALKING_GAME_REPORT.md docs/walking_game_report.pdf
```

The code, results and this report are released under the MIT license.